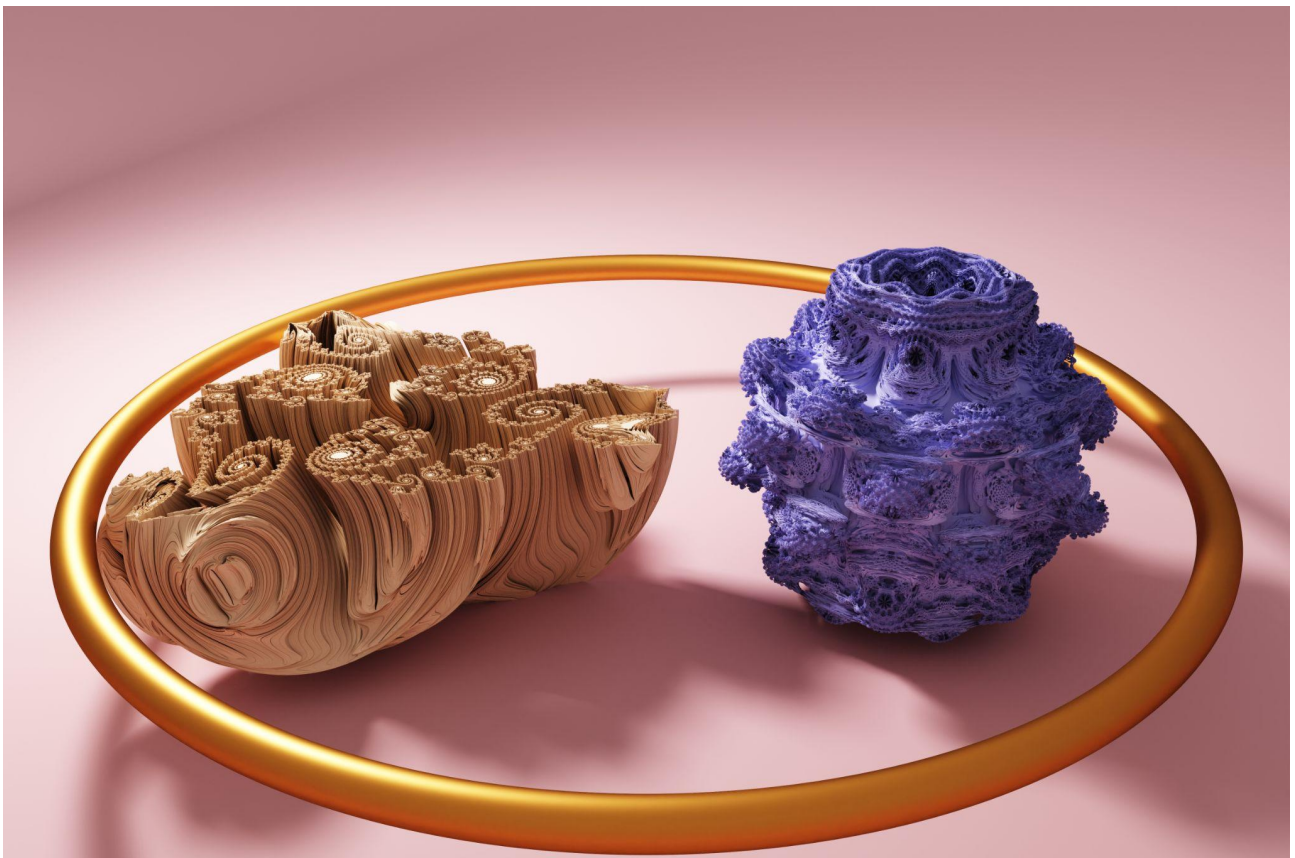


# Synthèse d'image réaliste

Rapport de Projet - TP



# Sommaire

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>Sommaire</b>                                       | <b>2</b>  |
| <b>Introduction</b>                                   | <b>3</b>  |
| <b>Les TPs</b>                                        | <b>3</b>  |
| TP1 - Ray casting                                     | 3         |
| Image, caméra et rendu                                | 3         |
| Sphère et intersection                                | 3         |
| Antialiasing                                          | 4         |
| TP2 - Éclairage et calcul d'ombres portées            | 4         |
| Lumière et ombrage                                    | 4         |
| Intégrateur                                           | 4         |
| TP3 - Sources lumineuses surfaciques et ombres douces | 4         |
| Quad Light                                            | 4         |
| Multi-sampling des ombres                             | 5         |
| TP4 - Matériaux et BRDFs                              | 5         |
| Lambert                                               | 5         |
| Oren-Nayar                                            | 5         |
| Phong et Blinn-Phong                                  | 5         |
| Cook-Torrance                                         | 5         |
| TP5 - Réflexions, réfractions                         | 6         |
| Intégrateur Whitted                                   | 6         |
| Miroir                                                | 6         |
| Transparent                                           | 6         |
| TP6 - Triangle, maillage et BVH                       | 7         |
| Maillage                                              | 7         |
| Boite englobante                                      | 8         |
| BVH                                                   | 8         |
| TP7 - Surface implicite et sphère tracing             | 8         |
| Sphere tracing                                        | 8         |
| SDF                                                   | 9         |
| Améliorations                                         | 9         |
| <b>Fonctionnalités supplémentaires</b>                | <b>10</b> |
| Architecture                                          | 10        |
| Scene setup                                           | 10        |
| Renderer settings                                     | 10        |
| Propriétés des rayons                                 | 10        |
| Lumières                                              | 10        |
| Simple quad light                                     | 10        |
| Sphère Light                                          | 11        |
| Sun Light                                             | 11        |
| Matériaux                                             | 12        |
| PBR                                                   | 12        |
| Sampling & pdf                                        | 13        |
| Émission                                              | 13        |

|                                  |           |
|----------------------------------|-----------|
| Rendu                            | 13        |
| Path-Tracing                     | 13        |
| Next event estimation            | 14        |
| Stratified Sampling Antialiasing | 15        |
| Tone mapping                     | 15        |
| Textures                         | 16        |
| Classe Textures                  | 16        |
| Sur les modèles                  | 16        |
| Textures procédurales            | 16        |
| <b>Annexes</b>                   | <b>17</b> |

# Introduction

Ce projet d'UE consiste à créer un moteur de rendu réaliste simulant la lumière par lancé de rayons.

Ce projet fut pour moi l'une des meilleures expériences pédagogiques de mon parcours, nous allons explorer les fonctionnalités implémentées durant les TP, ainsi que les améliorations apportées.

## Les TP

### TP1 - Ray casting

Ce tp consiste à lancer des rayons depuis une caméra et à intersecter la scène puis afficher les données dans une texture.

#### Image, caméra et rendu

On commence par parcourir chaque pixel de l'image dans la boucle de rendu. En fonction de la coordonnée du pixel dans l'image.

La caméra permet de créer une grille dans laquelle on peut lancer des rayons. La position de la caméra est l'origine du rayon, et en fonction de la position du pixel courant et des propriétés de la caméra, on peut déterminer la direction du rayon.

Une fois les rayons obtenus, on lance un rayon par pixel et on affiche les valeurs de la direction du rayon.

#### Sphère et intersection

La sphère est un objet qui contient une géométrie, la géométrie fournit le test d'intersection, que l'objet sphère passe ensuite à la scène. À chaque rayon lancé, la scène itère parmi tous les objets afin de déterminer potentiel le point d'intersection entre le rayon et l'objet. Si une intersection est trouvée, on remplit l'objet Hitrecord.

Le HitRecord est ensuite utilisé dans l'intégrateur dans le but de déterminer quelles sont les propriétés de la lumière à ce point d'intersection. En l'occurrence, on pondère la couleur du matériau par le cosTéthà de la normale de l'objet.

L'intersection et la normale au point d'intersection sont fournis par la géométrie, pour la sphère, on utilise le code du TD de FIG du semestre 1.

## Antialiasing

Un seul rayon par pixel peut produire du crénelage. Par exemple, dans le cas de la sphère, la surface est continue, le fait de la discrétiser dans une grille n'est pas bon.

Pour intégrer correctement la surface projetée sur un pixel, il faut approximer l'intégrale dans une somme finie, en tirant plusieurs rayons par pixels.

On a une méthode multisampling() que l'on appelle dans la boucle de rendu et qui lance plusieurs échantillonnages <sup>1</sup> et calcule la moyenne pour un pixel. Les échantillons sont choisis aléatoirement à l'intérieur de la zone du pixel. Avec un nombre d'échantillons élevés, on s'approche de la valeur de la véritable intégrale.

Le second type d'antialiasing implémenté est détaillé dans la section [Stratified Sampling Antialiasing](#).

## TP2 - Éclairage et calcul d'ombres portées

### Lumière et ombrage

Dans le cadre ce moteur, une lumière est définie par des propriétés comme sa position, sa couleur, son intensité. Un LightSample est en quelque sorte un conteneur d'informations lumineuses pour un échantillon. Une lumière peut fournir un LightSample à partir de n'importe quel autre point dans la scène.

Pour calculer l'éclairage, si l'on a une intersection avec un objet de la scène, alors on interroge les lumières de la scène pour obtenir des informations qui sont transmises sous la forme d'un LightSample. Ce qui permet de connaître l'intensité lumineuse en ce point.

Pour savoir si un point est illuminé ou non, on lance un nouveau rayon de ce point vers les sources lumineuses. Si on intersecte un seul objet en chemin, alors cela signifie que le parcours de la lumière est bloqué et qu'elle ne doit pas être prise en compte.

La méthode intersectAny() est implémentée comme alternative à intersect(), la différence est que l'on a besoin de savoir s'il y a au moins une seule intersection, et non pas de chercher la toute première. Également, il n'y a pas de HitRecord à remplir. Cela fait qu'elle est plus rapide à calculer que intersect().

### Intégrateur

L'intégrateur DirectLight prends en compte les notions précédentes et calcul l'éclairage direct en un point à partir d'un rayon. L'intensité lumineuse est ici déterminée par la couleur de l'objet et pondérée par les propriétés du lightSample, comme la radiance, la couleur, la pdf (1 dans le cas de la PointLight car tous les échantillons sont tirés au même endroit).

Pour éviter les autointersections au moment du tirage des rayons d'ombrages, on déplace légèrement l'origine du rayon selon la normale de l'objet.

L'intégrateur a une méthode protégée \_directLighting() qui premetra aux autres intégrateurs héritant de calculer l'éclairage direct en un point.

## TP3 - Sources lumineuses surfaciques et ombres douces

### Quad Light

Dans la réalité, les lumières ponctuelles n'existent pas, un point singulier ne peut pas être source lumineuse. C'est pourquoi les lumières surfaciques sont plus adaptées à la simulation.

Dans le principe, on a une surface, que l'on peut échantillonner pour estimer l'intensité lumineuse en un point. À partir de deux vecteurs, on calcule la normale de la lumière et sa surface. La surface servira à déterminer

---

<sup>1</sup> On dira souvent "sample" à la place d'échantillon



la puissance finale de l'échantillon, les vecteurs servent à échantillonner un point et la normale sert à pondérer l'intensité selon l'angle.

## Multi-sampling des ombres

Pour obtenir des ombres précises et diminuer la variance, il faut échantillonner plus, afin de mieux approximer l'intégrale de l'hémisphère autour du point. On lance plus d'échantillons sur les lumières surfaciques, tous les échantillons arrivent sur des points différents et intersectent potentiellement différents objets en chemin. Cela produit des ombres douces.

## TP4 - Matériaux et BRDFs

Jusque-là, on utilisait des couleurs unies sur la surface des objets et seuls les changements d'intensité lumineuse faisaient varier cette couleur. Désormais, nous utiliserons des matériaux dont les propriétés participeront également à l'aspect de l'objet.

Chacun de ces matériaux avancés dispose d'une ou plusieurs BRDFs qui définissent le comportement de la lumière en fonction de l'angle d'observation et de l'angle incident par rapport à la normale de la surface.

Les matériaux prennent en entrée le rayon partant du point vers la source de lumière, le rayon d'observation, partant du point vers la direction d'observation (la caméra dans le cas de l'éclairage direct). Le hitrecord est passé aussi et permet au matériau de connaître les informations supplémentaires de l'intersection, comme les coordonnées UV au sein d'un triangle ou la position du point dans l'espace.

À la construction des BRDFs, on fournit les paramètres constants, pour l'évaluation, on a en entrée que  $w_i$ ,  $w_o$  ainsi que la normale et potentiellement d'autres paramètres fournis par le matériau comme la roughness en un point par exemple.

### Lambert

Le modèle de Lambert simule une surface qui diffuse parfaitement la lumière équitablement dans toutes les directions. On divise l'intensité de la couleur par une constante ( $1/\pi$ ). Si on intègre une lumière blanche provenant de toutes les directions, cette constante nous permet de retomber sur la couleur d'origine du matériau. Ainsi l'énergie est préservée et aucune nouvelle lumière n'est créée.

### Oren-Nayar

Le modèle d'Oren Nayar permet lui aussi de simuler des matériaux diffus, mais avec un aspect plus rugueux (et réaliste) que Lambert. Dans la BRDF, on doit effectuer un passage des coordonnées cartésiennes aux coordonnées sphériques afin de calculer correctement les composantes du matériau. Le facteur  $\sigma$  permet de contrôler le niveau de rugosité. Il représente en fait la quantité d'inclinaison dans le modèle statistique de micro-facettes sous-jacent.

### Phong et Blinn-Phong

Les modèles de Phong et Blinn-Phong sont des approximations empiriques du comportement des matériaux. Les BRDF sont calculées comme dans l'UE M3D du s1. Ils ont l'avantage d'être rapides à évaluer.

On additionne la composante diffuse fournie par le modèle de Lambert à la composante spéculaire. Dans le code, le passage d'un modèle à l'autre se fait avec un paramètre passé à la construction du matériau.

### Cook-Torrance

C'est un modèle basé sur les micro facettes qui permettent dans ce cas d'approximer les matériaux de type métaux ou conducteurs. Les réflexions sont teintées par la couleur du matériau.

Dans le code, le matériau est implémenté conjointement avec Oren-Nayar pour la composante diffuse et on réalise une interpolation entre les deux en fonction du niveau de brillance désiré. Cela ne représente pas un comportement physiquement basé, mais permet d'avoir aisément accès à une alternative intéressante à Phong.

# TP5 - Réflexions, réfractions

## Intégrateur Whitted

L'intégrateur est basé sur celui de l'éclairage direct. On parcourt récursivement la scène jusqu'à la condition d'arrêt. Si on rencontre un matériau susceptible de renvoyer ou transmettre la lumière (mirror et transparent dans ce cas), alors on envoie un nouveau rayon qui est réfléchi ou réfracté par la surface en fonction de l'angle d'incidence à la normale.

On s'arrête après un nombre maximum de rebonds qui est défini dans les options de rendu.

## Miroir

Le matériau miroir réfléchit parfaitement la lumière. On n'a pas besoin de pondérer l'intensité de la lumière réfléchie par un facteur, car la pdf est un diarc : il n'y a qu'une seule direction d'incidence possible pour calculer la totalité de l'intégrale sur l'hémisphère. On utilise simplement la fonction de réflexion de glm pour obtenir un nouveau rayon, que l'on lance.

## Transparent

Dans le cas de la transparence, il y a un peu plus de complexité. Déjà comme pour le miroir, on n'a pas besoin de pondérer la valeur de la réfraction, car il n'y a qu'une seule direction possible. On utilise la fonction de réfraction de glm pour calculer la direction de réfraction. La difficulté réside non seulement dans le fait que le rayon doit traverser la surface de l'objet, mais aussi parce qu'il faut lancer un rayon de réflexion également.

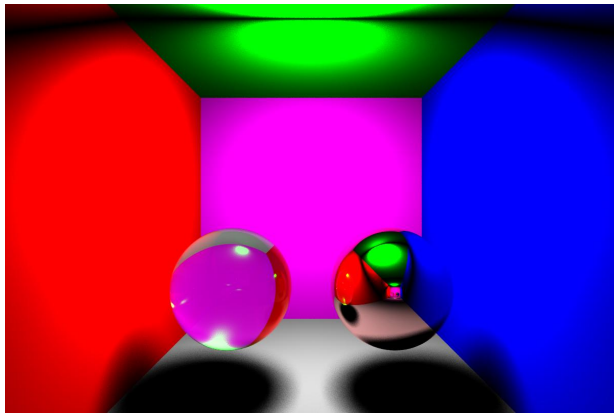
De plus le rayon réfracté doit être capable de traverser la surface et il faut faire attention à l'offset dans le bon sens. Et lorsque l'on se trouve à l'intérieur de la surface, les coefficients  $n_1$  et  $n_2$  des équations de Fresnel se retrouvent inversés. Si on prend correctement en compte tous ces paramètres, le cas de la réflexion interne totale devrait se gérer seul.

Dans l'intégrateur, j'ai ajouté deux méthodes supplémentaires afin de réfracter et de réfléchir les rayons sans surcharger la méthode `trace()`.

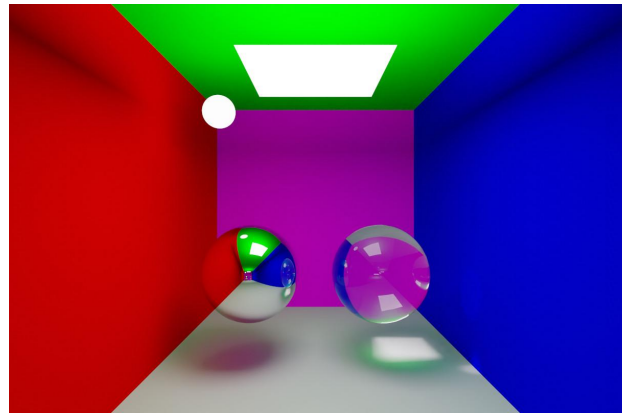
En réalité, le matériau transparent est une interpolation entre un matériau miroir et un matériau de réfraction pure, ce mélange est pondéré par le facteur de Fresnel, qui détermine la chance de réflexion.

On peut noter que 2 rayons peuvent simultanément être lancés dans le cas de la transmission. Ce qui a pour conséquence de faire augmenter exponentiellement le temps de rendu dans les scènes où il y a beaucoup d'objets réfractant. Pour remédier à ce problème, on peut considérer le facteur de Fresnel comme une chance de réflexion et réfraction. Ainsi, avec un nombre de samples élevé et un seul rayon réfléchi ou réfracté, le résultat convergera vers la valeur attendue, mais produira du bruit. En contrepartie, on a un temps de calcul qui croît linéairement avec le nombre de samples par rayon.

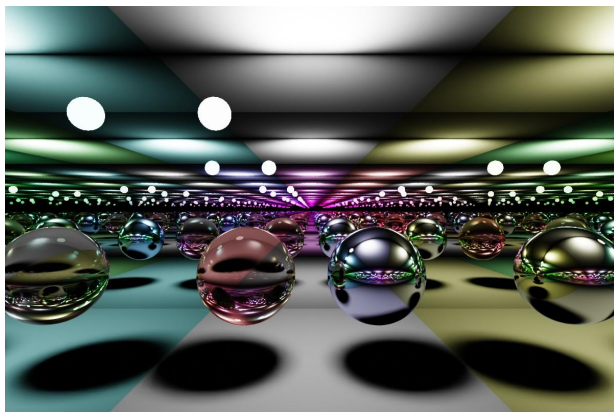
Pour teinter les réflexions et réfractions, on fait hériter les matériaux transparents et les matériaux miroir de la classe `ColorMaterial` et on multiplie le résultat du rebond par la couleur.



Whitted integrator et quad light



Mesh light et path tracing (caustics!)



La sphère de gauche est du verre teinté rouge clair, celle de droite est un miroir teinté bleu clair. Les murs sont des miroirs teintés cyan à gauche, jaune à droite, blanc devant et magenta derrière. Le plafond et le sol sont des matériaux matte.



Un matériau miroir et un matériau transparent appliqués au mesh Bunny et à une UV sphère.

## TP6 - Triangle, maillage et BVH

### Maillage

Un maillage est composé références vers des triangles et chaque triangle référence son maillage. On utilise Assimp pour charger les modèles.

L'intersection entre un rayon et un triangle se fait en déplaçant l'origine du rayon puis en le projetant dans l'espace du triangle selon cette méthode de Möller et Trumbore<sup>2</sup>. Ainsi, on peut savoir si oui ou non le triangle a été intersecté, mais également les coordonnées uv de l'intersection à l'intérieur du triangle. Ce qui pourra être utile plus tard pour les normales et les coordonnées de textures.

Pour l'intersection d'un maillage avec un rayon, on itère sur tous les triangles, et on garde en mémoire seulement les propriétés de l'intersection la plus proche de l'origine du rayon. Ensuite, si on a trouvé une intersection, on remplit le HitRecord avec ces propriétés.

<sup>2</sup> <https://cadvfem.org/inf/Fast%20MinimumStorage%20RayTriangle%20Intersection.pdf>

## Boîte englobante

Les AABB sont des boîtes englobantes de la géométrie. On peut les construire en spécifiant les positions min et max. On étend un AABB avec les fonctions `glm::min` et `glm::max` qui donnent le minimum des coordonnées de deux points. Et on peut étendre depuis un triangle, en itérant sur les 3 vertices.

Ainsi, on peut créer un AABB englobant un maillage en étendant l'AABB avec tous les triangles de l'objet au chargement. Ensuite, pendant le rendu, on peut tester l'intersection avec l'AABB avant de tester l'intersection avec les triangles pour savoir si le rayon a une chance de toucher l'objet.

Pour l'intersection avec un AABB, j'ai d'abord implémenté une méthode naïve de test d'intersection avec six plans en vérifiant que les coordonnées sont dans la boîte. Ensuite j'ai implémenté quelques optimisations du fait que les délimitations de la boîte sont alignées aux axes. Les résultats étaient satisfaisants. Ensuite j'ai implémenté une version plus optimisée de ScratchPixels basé sur la méthode de Amy Williams <sup>3</sup>.

Mes méthodes sont évidemment moins performantes que la nouvelle implémentée, Avec Bunny.obj seul dans la scène, on observe une différence d'environ 25 %.

## BVH

Le BVH est composé d'AABB organisés dans un arbre. Au niveau des feuilles, on a également les triangles qui composent le maillage. Chaque nœud du bvh contient des références vers ses fils, son propre AABB, et les indices des triangles qui le compose.

On construit récursivement le BVH en assignant les indices du premier et du dernier indice de triangles présents dans le nœud. Puis, on construit l'AABB du nœud en ajoutant tous les triangles. Ensuite, on trie les triangles selon le plus grand axe de l'AABB avec la méthode `std::partial_sort()`. Pour le trie des triangles, chaque triangle possède son propre centroïde, qui est calculé au moment de la création du triangle (cela permet d'éviter d'avoir à enregistrer un AABB par triangle). Ensuite, les triangles sont répartis entre les nœuds de droite et de gauche, on coupe toujours au milieu pour le moment. Si on arrive à la condition d'arrêt qui est déterminé par la profondeur ou le nombre de triangles, alors on calcule seulement l'AABB.

Dans `intersectAny()`, on peut bénéficier de la lazy évaluation et renvoyer l'intersection du fils gauche ou du fils droit dans la même ligne pour éviter de parcourir trop l'arbre.

## TP7 - Surface implicite et sphère tracing

### Sphère tracing

Le sphère tracing est une méthode proposée par John Hart <sup>4</sup> pour intersecter des géométries implicites. Dans le principe, on marche le long du rayon selon une distance qui est déterminée par la fonction de distance de la surface. Si la distance est assez faible, on considère l'intersection comme trouvée et on remplit le HitRecord.

On utilise une variable `t` que l'on initialise à `tmin` et que l'on incrémente selon la distance parcourue jusqu'à `tmax`. La condition d'arrêt est sur `t`, mais aussi sur le nombre maximum de pas effectués. C'est pour ne pas passer trop de temps dans les détails minimes.

Pour l'évaluation des normales, on utilise le gradient en un point. La direction inverse du gradient donne la direction de la normale. Pour connaître le gradient, on évalue plusieurs fois la sdf autour du point sur les

---

<sup>3</sup> An Efficient and Robust Ray-Box Intersection Algorithm. Amy Williams et al. 2004. Et <https://www.scratchapixel.com/lessons/3d-basic-rendering/minimal-ray-tracer-rendering-simple-shapes/ray-box-intersection> (sérieusement, ce site est une ressource complémentaire incroyable !)

<sup>4</sup> John Hart. Sphere tracing : A geometric method for the antialiased ray tracing of implicit surfaces. The Visual Computer, 12, 06 1995.

axes selon un décalage minime. On utilise la méthode proposée par Inigo Quilez <sup>5</sup>. On peut noter également qu'il est intéressant d'augmenter la taille de ce décalage lorsque la taille de la surface au point que l'on approxime augment pour diminuer l'aliasing (en fonction de t par exemple). Mais comme on est en rendu off, l'intérêt est limité.

On utilise des doubles au lieu des floats dans l'algorithme, pour gagner en précision dans l'affichage, mais aussi pour éviter les problèmes liés aux distances extrêmes <sup>6</sup>.

## SDF

Pour déterminer la distance à parcourir dans le tracé de sphères, chaque objet implicite fournit une fonction de distance signée qui permet de connaître la distance entre la surface de l'objet et un point. Les implémentations des SDF sont variées.

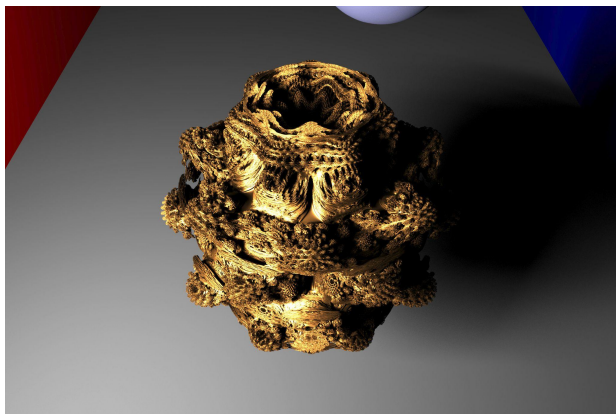
J'ai implémenté la sphère, le tore, le Mandelbulb, et le set de Julia selon les sdf de Inigo Quilez. Le tore et la sphère utilisent des mathématiques simples. Tandis que les deux fractals donnent la sdf en itérant dans une boucle interne pour obtenir les répétitions infinies.

Toutes les sdf sont évaluées en  $[0,0,0]$ , nous allons ensuite voir comment changer leur position.

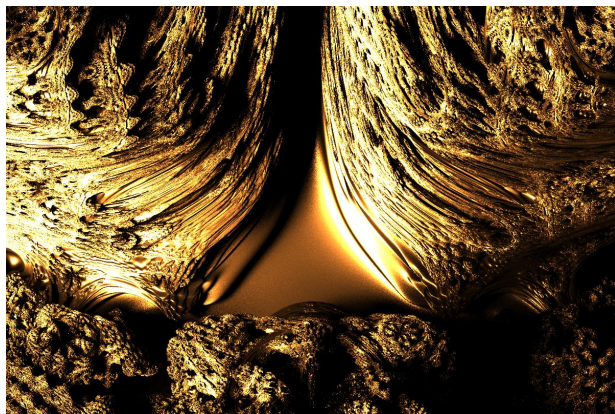
## Améliorations

Dans la classe de base des surfaces implicites, on a un attribut de position et un facteur d'échelle. Toutes les sdf peuvent être translatées ou redimensionnées selon ses facteurs. Pour cela la fonction publique `sdf()` décale le point à évaluer selon la position inverse et effectue un appel à la fonction protégée `_sdf(p / scale) * scale`; Où p est le point. Dans les classes dérivées, c'est la méthode `_sdf()` qui est redéfinie. Ainsi, on peut placer n'importe quel objet implicite là où l'on souhaite dans la scène.

La classe de base propose aussi un AABB afin d'accélérer les tests d'intersections et éviter de faire marcher des rayons. Seulement cet AABB doit être manuellement remplis pour chaque nouvelle surface implicite, en effet il est *difficile* d'estimer un AABB pour une sdf quelconque. Pour les 4 fournies, j'ai fait en sorte que l'AABB soit correctement créé dans le constructeur. Également, l'AABB est correctement déplacé en fonction de la taille et de la position des surfaces dans la scène.



Mandelbulb, éclairage direct



Mandelbulb, zoomé

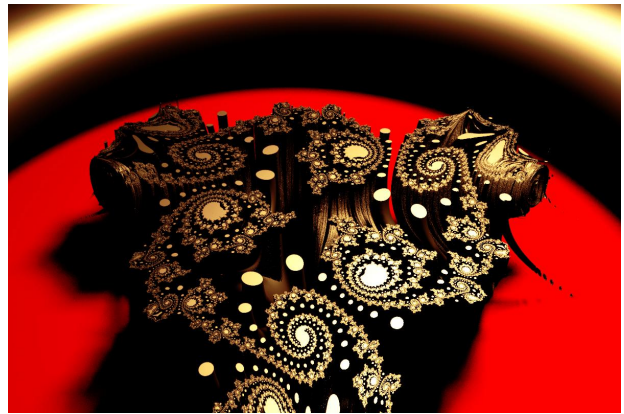
---

<sup>5</sup> <https://iquilezles.org/articles/normalSDF/>

<sup>6</sup> Un problème que j'ai rencontré est lorsque qu'une quadlight est perpendiculaire à un plan, les rayons rasants qui intersectent proche de la taille maximale du float avaient du mal à revenir jusqu'à l'origine de la scène lors du calcul de l'éclairage direct.



Set de julia coupé en deux, éclairage indirect



Set de julia coupé en deux, éclairage direct

## Fonctionnalités supplémentaires

### Architecture

#### Scene setup

Pour naviguer plus facilement entre les TP et pouvoir faire des tests, il y a un manager de scènes. On peut initialiser une scène depuis la fonction main en fournissant l'enum correspondant.

C'est Maintenant la scène qui donne une caméra au Renderer.

#### Renderer settings

Les options de rendu sont gérées dans une structure qui est passée au Renderer à son initialisation. On y trouve notamment la spécification du format de sortie, le nombre d'échantillons, le type d'échantillonnage, le nombre de rebonds pour les différents types de rayons et l'intégrateur à utiliser.

#### Propriétés des rayons

Désormais les rayons lancés contiennent leurs propres propriétés, comme leur origine (caméra, rebond, réflexion, ...), le nombre de rebonds déjà effectués, les valeurs tmin, tmax.

Ainsi, on peut par exemple contrôler indépendamment le nombre de rebonds diffus, transmis et réfléchis. Également, cela évite d'avoir à passer en paramètres toutes ces informations à chaque récursion.

### Lumières

#### Simple quad light

Une classe dérivée SimpleQuadLight est disponible, ici, on spécifie seulement la position et la surface, ainsi qu'un paramètre lookAt. À partir duquel on crée une nouvelle lumière surfacique carrée, dont le centre se trouve à la position fournie, et dont la normale pointe dans la direction demandée. Les vecteurs U et V sont orthogonaux à la normale. Cette lumière est plus simple d'utilisation, car elle reste toujours centrée sur sa position et est aisément orientable.

Cela permet de gérer plus naturellement le positionnement de ce type de sources lumineuses.

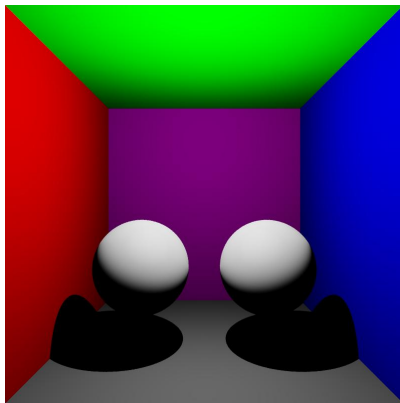
De plus, l'intensité ne dépend plus de la surface, ainsi c'est plus simple de régler le niveau de luminosité lorsqu'on veut des ombres très douces et donc une grande taille. La valeur par défaut a été calibrée pour correspondre à l'intensité lumineuse d'une PointLight.



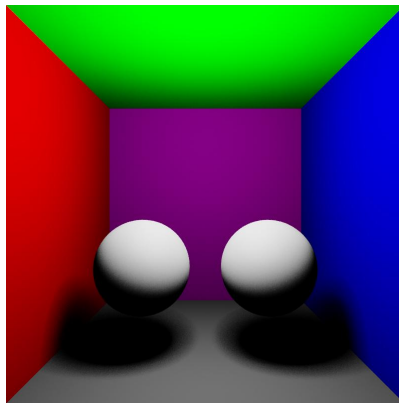
L'aire n'entre en conséquence pas en compte dans le calcul de la radiance et de la pdf. C'est valide dans ce cas, car l'air est une constante et est de ce fait la même pour la pdf de tous les échantillons sur la surface.

## Sphère Light

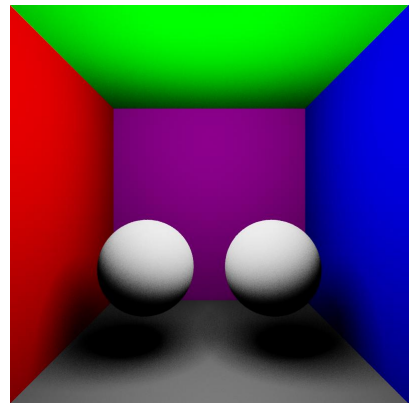
On reprend la classe PointLight à laquelle on ajoute un radius. Si le radius est sous un certain seuil, on considère que la lumière est un seul point. Sinon la lumière est considérée comme une sphère. Pour obtenir un échantillon sur la surface, on prend un vecteur aléatoire dans l'hémisphère qui se trouve dans la direction du point qui échantillonne. On calcule la pdf avec  $\cos\theta$  et la normale. Le rayon de la sphère n'entre pas en compte dans le calcul de la radiance ou de la pdf, ainsi il est plus simple de contrôler l'intensité comme pour la simple quad light.



sphère light, size = 0



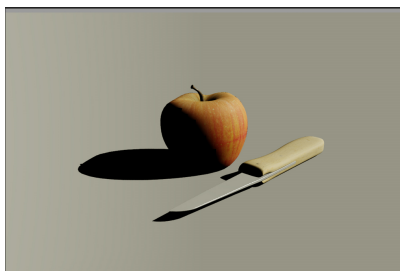
sphere light, size = 0.5



sphere light, size = 1

## Sun Light

La lumière de type soleil se comporte comme une lumière distance à l'infini. Tous les samples ont une distance maximale et la même direction. On peut cependant produire des ombres douces en changeant la taille, alors, les samples auront des directions légèrement différentes. La pdf est constante en tout point, et la distance n'entre pas en compte dans le calcul de l'intensité.



sunlight, size = 0



sunlight, size = 0.1



sunlight, size = 0.3

## Matériaux

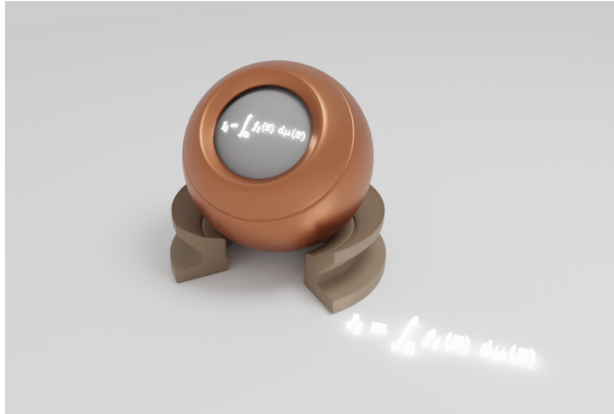
### PBR

Le matériau PBR simule un matériau réaliste. On utilise le workflow de metalness. Au lieu de valeurs spéculaires, on a une valeur de rugosité et un IOR <sup>7</sup>. Une composante réfléchissante est mélangée à une

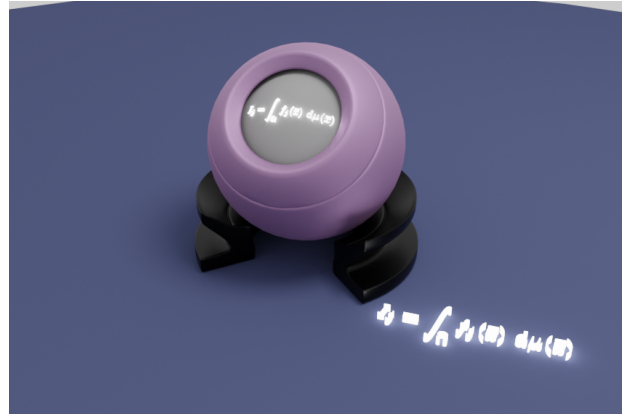
<sup>7</sup> La plupart des matériaux diélectrique ont un ior autour de 1.4. C'est la valeur de cet ior qui contrôle maintenant l'intensité des réflexions.

composante diffuse selon un facteur de Fresnel. De plus, la quantité de la diffuse augmente avec la rugosité du matériau. Les réflexions ne sont pas teintées, car c'est comme ça que se comportent la plupart des diélectriques.

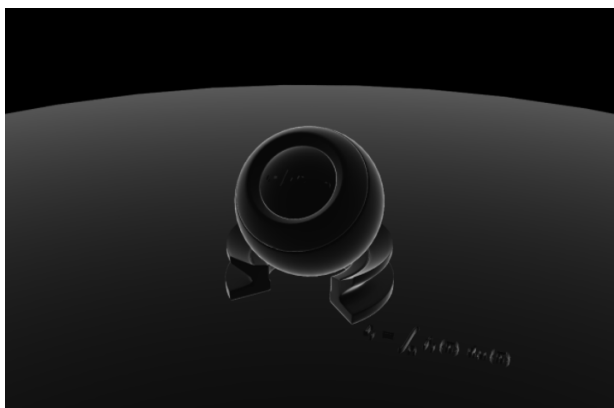
Également pour les matériaux conducteurs et métaux (tous les matériaux non diélectriques), le comportement change avec la valeur de metealness. Maintenant les réflexions sont teintées avec la couleur du métal. Mais il y a tout de même une perte de la saturation aux angles rasants.



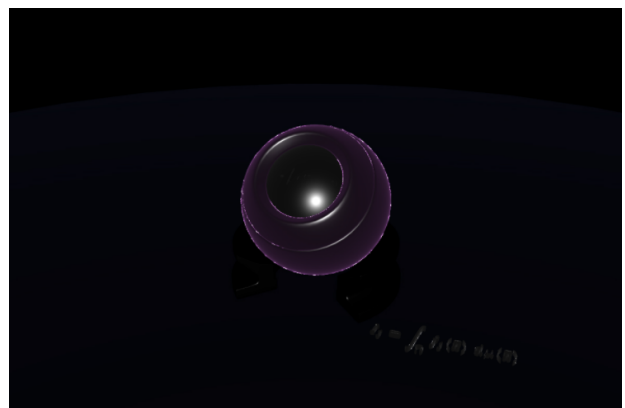
Rendu PBR, le texte est émissif, le socle est un matériau diffus de type carton et l'orbe est un métal type cuivre.



Même rendu, mais avec des matériaux différents.

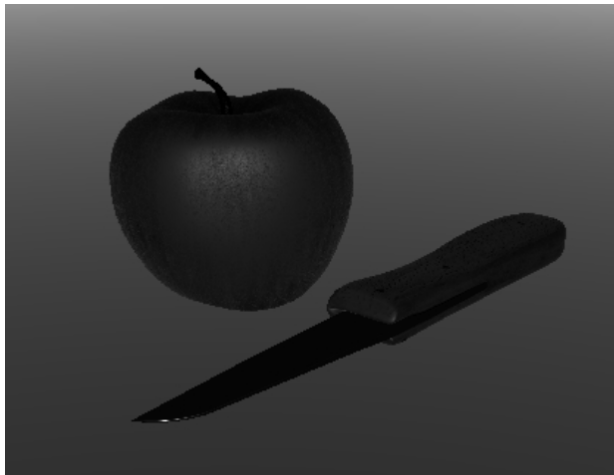


Le facteur de Fresnel qui pondère le mélange des brdfs. Ici la roughness est mise à 0.0 pour tous les matériaux.

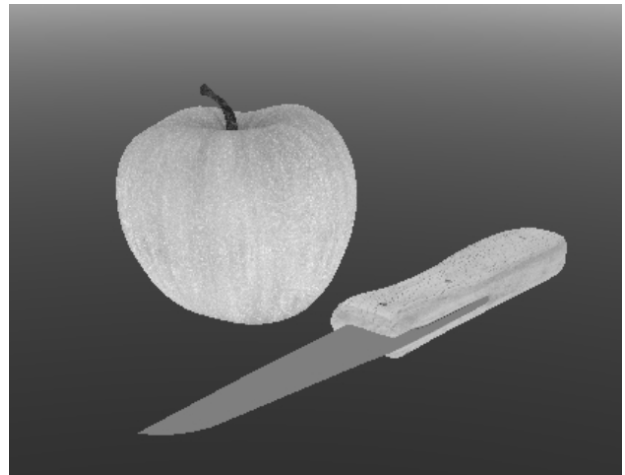


Couleur rendue par shade(), on remarque le dégradé vers le blanc aux angles rasants sur la partie métallique





rendu uniquement de la partie glossy du pbr, plus



Affichage des cartes de roughness, plus la couleur est sombre, plus le matériau apparaît est réfléchissant

## Sampling & pdf

On peut faire en sorte que les brdf fournissent une direction à sampler, ainsi que la pdf correspondante. Dans le matériau, si on a plusieurs brdf, on mélange le résultat des samples de la même manière que les brdf sont implémentées mélangées.

## Émission

Un matériau émissif permet de produire de la lumière dans le cas de l'éclairage indirect. Et permet également de donner une couleur qui n'est pas impactée par le reste de l'environnement.

Pour cela, une méthode `getEmit()` a été ajouté à la classe `BaseMaterial`.

## Rendu

### Path-Tracing

Le nouvel intégrateur `PathTracingIntegrator` qui hérite de `Whitted` permet de prendre en compte les rebonds diffus de la lumière.

Dans le principe, à chaque intersection, on teste le type de matériau, s'il est transparent ou miroir pur, le cas est géré par `Whitted`, s'il est diffus, alors on échantillonne une nouvelle direction. On trace un nouveau rayon dans cette direction et on multiplie la couleur au point actuel par la luminance provenant du rayon que l'on vient de lancer.

On s'arrête lorsqu'on a atteint le nombre maximum de rebonds autorisé pour ce type de rayon, ou lorsqu'on atteint l'arrière-plan de la scène.

Pour la luminance en un point, on applique simplement l'équation de la lumière avec l'intensité incidente pondérée par  $\cos\theta$  et la valeur de la brdf en ce point, et divisé par la pdf<sup>8</sup>.

Pour les sources lumineuses, on n'évalue plus les lumières, mais on utilise un nouveau matériau émissif qui permet d'illuminer les rayons qui y arrivent.

---

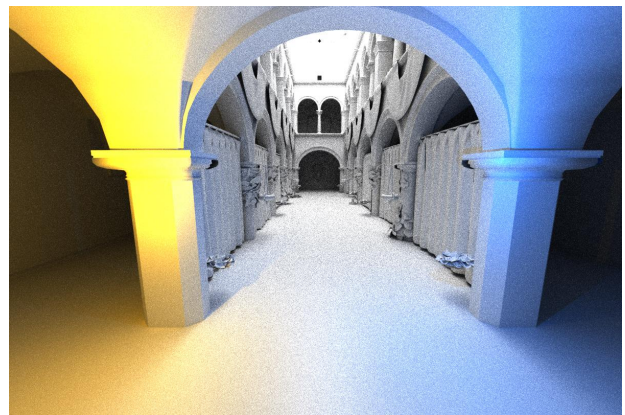
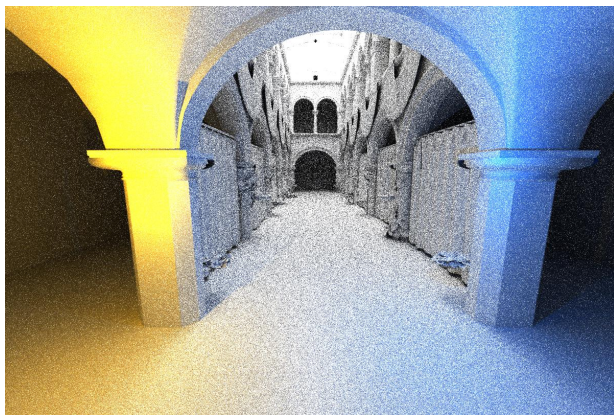
<sup>8</sup> La Pdf est déterminée en fonction du type d'échantillonnage, elle permet de corriger mathématiquement l'intégration de Monte Carlo lorsque les samples sont pondérés dans une direction (importance multiple).

Pour la direction d'échantillonnage, on essaye d'intégrer l'hémisphère autour du point, on a plusieurs choix :

- Échantillonnage naïf uniforme : une direction aléatoire dans l'hémisphère de la normale. Alors la pdf est également uniforme et prend la valeur  $\frac{1}{2\pi}$ .
- Pondéré par  $\cos\theta$  : On utilise un nombre aléatoire pour obtenir un sample qui a plus de chance d'être dans la direction de la normale (intéressant pour les matériaux diffus) et ce nombre permet aussi d'obtenir la pdf correspondante.
- Donnée par la brdf, dans ce cas la pdf est pareillement donnée par la brdf.

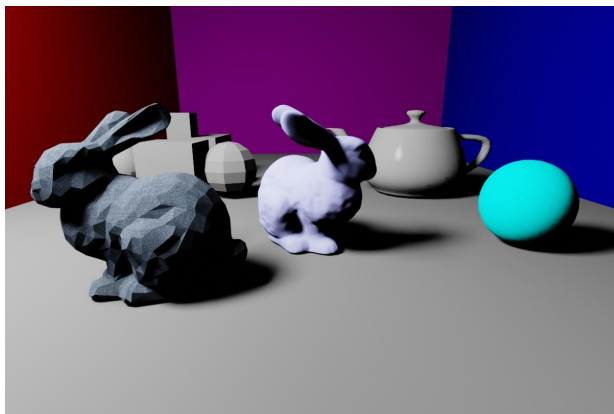
Pour deux premiers types d'échantillonnage, je calcule les tangentes et bi-tangentes à la normale afin d'avoir un nouveau repère orthonormé dans lequel on peut émettre de nouveaux vecteurs. On aurait aussi simplement pu tirer un `Vec3f` aléatoire et l'inverser s'il n'est pas dans la direction de la normale.

Notons que l'intersection avec une surface émettrice n'est pas une condition d'arrêt ici. En effet, les surfaces lumineuses peuvent posséder une composante diffuse qui peut être visible dans le cas où l'émission est faible, ou si elle-même reçoit une plus forte intensité lumineuse d'une source voisine.



100 spp (éclairage seulement par l'environnement)      1000 spp (éclairage seulement par l'environnement)

Comme on peut le voir sur les images ci dessus, avec la méthode naïve d'échantillonnage, il faut un très grand nombre de samples avant que la variance ne devienne acceptable.



Éclairage direct



Éclairage indirect

## Next event estimation

Les lumières directes n'étaient pas prises en compte dans l'éclairage par path tracing. Pour remédier à cela, on calcule désormais l'éclairage direct à chaque rebond, et en fin de récursion. En plus de réduire considérablement la variance, cela permet de prendre en compte les lumières de la scène.



Éclairage direct



Éclairage indirect et next event estimation.

## Stratified Sampling Antialiasing

Un deuxième type d'anticrénelage est disponible : le stratified sampling. Dans ce cas, le nombre d'échantillons par pixel est élevé au carré et on distribue les samples dans une grille. La position exacte des échantillons est tirée aléatoirement dans chaque case de la grille. Ainsi, on réduit la variance en échantillonnant de manière plus intelligente. Cela évite que tous les échantillons soient regroupés dans un coin d'un pixel par exemple.

## Tone mapping

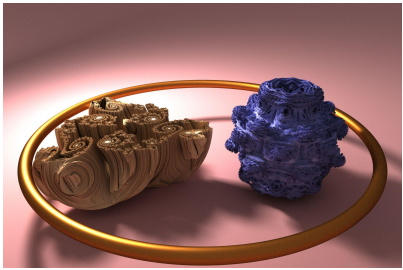
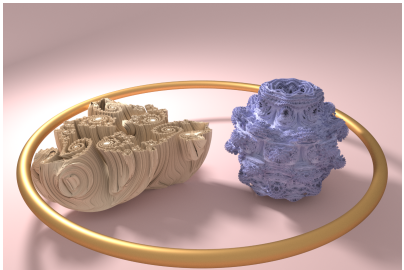
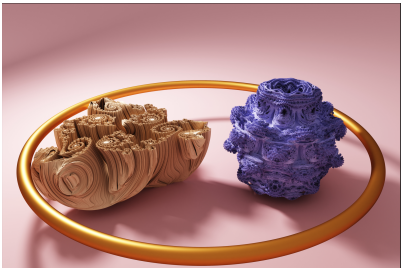
Le moteur de rendu actuel ne calcul pas de couleur, mais une radiance (ou intensité) lumineuse. Dans les TP on a choisi de considérer une courbe de réponse linéaire de la radiance vers les couleurs RGB dans l'espace  $[0..1]$ . Mais on peut très bien envisager de transformer ces intensités en couleur autrement, afin d'éviter les problèmes de clipping au blanc et augmenter la plage dynamique.

Dans le renderer, avant d'écrire le pixel dans la texture, on applique une mise en correspondance tonale simple.

On utilise la transformation colorimétrique de ACESfilm<sup>9</sup> sur la luminance seulement. Une correction du gamma peut également être appliquée dans le cas où on ne souhaite pas appliquer de tone mapping mais juste avoir une image proche du style initial du moteur.

De plus, les options de rendu offrent la possibilité de choisir ou non de sauvegarder une image HDR en plus de celle de base. Dans le cas où un traitement dans un logiciel externe est désiré sans perte d'informations (pour de la correction artistique ou du denoising par exemple).

<sup>9</sup> [https://en.wikipedia.org/wiki/Academy\\_Color\\_Encoding\\_System](https://en.wikipedia.org/wiki/Academy_Color_Encoding_System)

|                                                                                   |                                                                                   |                                                                                     |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  |  |  |
| Linéaire                                                                          | sRGB                                                                              | Tonemapped                                                                          |

## Textures

### Classe Textures

initialement, la classe texture propose d'enregistrer les pixels avec des Octets. Ce format a été changé pour des flottants. La conversion en flottant est faite au chargement et la reconversion en octet est faite à l'enregistrement, avec un clamp dans 0..1. Ainsi, on peut stocker des intensités de couleurs au-delà des limites par défaut et enregistrer des images à haute plage dynamique.

Des accesseurs à la couleur d'un pixel, ainsi qu'à son alpha ont été ajoutés également.

### Sur les modèles

Lors du chargement d'un objet, si le matériau spécifie qu'une texture est disponible, alors on la charge dans un nouveau ColorMaterial. Pour cela, on récupère simplement le nom de la texture depuis Assimp et la classe texture se charge de la lecture avec STBI.

Pour afficher la texture sur un maillage, il nous faut connaître les coordonnées UV du point d'intersection dans le maillage. Ces coordonnées sont généralement enregistrées entre 0 et 1, mais rien n'empêche un maillage d'avoir des coordonnées supérieures pour répéter plusieurs fois la texture.

Pour récupérer ces coordonnées et évaluer la couleur de la texture, la routine d'intersection avec un triangle permet de savoir les coordonnées du point au sein d'un triangle. Ainsi, il suffit d'ajouter ces coordonnées aux coordonnées de la position du triangle dans la carte UV.

Pour récupérer le pixel en question dans la texture, on multiplie u et v par la hauteur et la largeur de la texture en prenant soin de boucler dans les cas où on tombe à l'extérieur.

### Textures procédurales

Les coordonnées des textures procédurales sont données par la position du point dans l'espace. On peut également utiliser la position dans l'espace relative à la position, la rotation et le facteur d'échelle de l'objet si l'on souhaite que la texture se déplace avec l'objet.

Pas eu le temps d'implémenter :/





Scène texturée



Même scène, mais sans textures

## Notes finales

J'ai presque tout codé dans des fichiers hpp par ce que je trouvais ça plus simple pendant le développement malgré les temps de compilation plus longs, malheureusement je n'ai pas eu le temps de séparer le code et les définitions en fin de projet.

J'aurais aimé ajouter plus de structures accélératrices, car je ne les ai finalement pas touchées du tout.

Une meilleure méthode de sampling pour l'intégration de la lumière à partir des brdf.

Le filtrage des textures, assez simple finalement, mais encore une fois pas eu le temps.

Dans le repo, dans le dossier result/fails, on peut trouver la galerie des horreurs, où tous mes plus mauvais résultats sont enregistrés.